TraceFlow 1.0.0: Validation Pack

November 18, 2025

Version under test	
Engineer	
Tester	
Test Date	
Initial Report Generation Date	November 18, 2025
Result	
Approved Usage	

Manual tests carried out by:

Signature

Name

Job Title

Date

Validation pack result approved by:

Signature

Name

Job Title

Date

Contents

1 Design - Functional Design Spec

This document provides a high-level architectural view of the TraceFlow demo and links each design decision to the requirements, tests, and clinical safety risks that drive them. Refer to **RISK-001** when assessing identifier safety controls described below.

1.1 Architecture overview

The system is split into ingestion, orchestration, and presentation tiers in order to satisfy **REQ-001** through **REQ-004** while maintaining a clean boundary for audit logging (**REQ-005**).

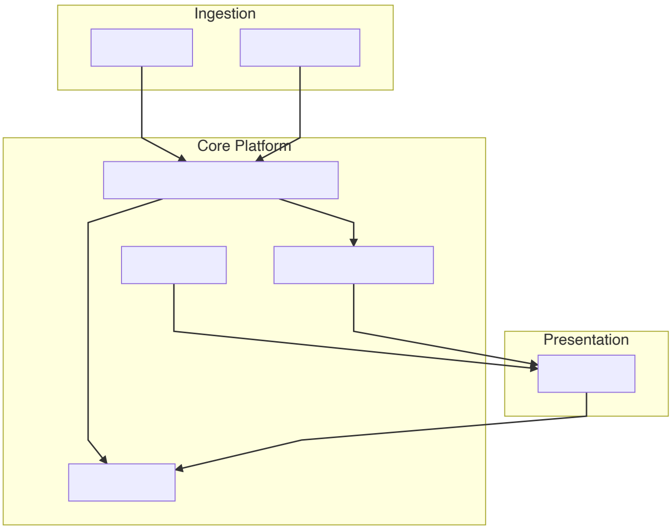

Figure 1:

1.2 Identifier reconciliation engine (REQ-005, RISK-001, TEST-002)

The reconciliation engine protects against the hazardous situation captured in **RISK-001** where patient and study identifiers might be mismatched. Design goals:

- Compare the DICOM `PatientID` and `AccessionNumber` with the HL7 metadata stream.
- If mismatches are detected, the transaction is quarantined and **TEST-002** exercises the operator override.
- Successful transactions emit structured audit events so **REQ-005** is met.

1.2.1 UML activity view

- Receive the incoming DICOM objects and perform schema checks.
- Compare HL7 metadata with extracted tags and branch depending on whether identifiers match.
- Persist successful studies and emit reconciliation metrics for **RISK-001** .
- Quarantine mismatches and require an operator acknowledgement that is captured by **TEST-002** .

1.3 Authentication and authorization (REQ-001, TEST-001)

Authentication relies on OpenID Connect so automated tests (**TEST-001**) can exercise both success and failure paths. Tokens carry the clinician's study permissions, and the middleware enforces them for every API endpoint.

1.4 Imaging pipeline (REQ-002, REQ-003, RISK-002)

The MRI import module normalizes DICOM before invoking the analysis pipelines described in **REQ-003**. The pipelines emit validation artifacts:

- DICOM specific logs for regulatory review.
- Image-derived measurements tied to their source requirement/test pairs.
- Alerts for saturation or out-of-range values, contributing to **RISK-002** mitigations.

1.5 Operational documents (General IUS)

Operational guidance for installations is maintained in `docs/ius.md`. That supplemental document links directly to **TEST-003** for continuous integration evidence and references **RISK-002** to remind the operator of the residual hazards.

2 Installation & User Specification

This generic operational document demonstrates how TraceFlow can include arbitrary Markdown artifacts that link back to requirements, tests, and risks.

2.1 Installation checklist

- Deploy the ingestion stack and confirm that the `traceflow` service exposes `/healthz` .
- Verify that authentication (**REQ-001**) succeeds for a clinical account with the `clinician` role by following **TEST-001** .
- Enable the identifier reconciliation engine (**REQ-005**) and record the resulting audit entry.
- Capture Playwright evidence from **TEST-003** and attach it to the release ticket.

2.2 Operational monitoring

- Operators review the reconciler dashboard hourly to ensure the guardrails for **RISK-001** and **RISK-002** remain in place.
- When a mismatch occurs, follow the escalation steps documented in the clinical SOP and reference the impacted **REQ-005** controls.
- Residual risks shall be re-evaluated whenever a new modality is onboarded.

2.3 Traceability table

Step	Linked Requirement/Test	Linked Risk
Configure audit webhooks	REQ-005	RISK-001
Re-run integration suite after upgrade	TEST-003	RISK-002
Attach validation pack to submission	TEST-001 / TEST-002	RISK-003

2.4 Sign-off

The implementation owner confirms that installation and operational verification were executed per procedure and that the TraceFlow validation pack is archived with the evidence references listed above.

3 Risk Register

TraceFlow libraries track software hazards alongside the requirements and tests that mitigate them.

Risk ID	Hazardous Situation	Harm	Cause	Risk (Severity × Probability)	Controls	Residual Risk
RISK-001: Incorrect study-patient association	Clinician views the wrong patient's images believing they are correct.	Misdiagnosis or inappropriate treatment.	Race condition during HL7/DICOM message processing leading to incorrect PatientID or AccessionNumber.	High × Medium = High (12)	Identifier reconciliation service (REQ-005) exercised by TEST-002.	Medium × Low = Low (3) Operator review plus automatic quarantine when mismatches occur.
RISK-002: Undetected imaging artifacts	Poor quality MRI acquisitions flow through the analysis pipeline without alerts.	False clinical conclusions or delayed treatment.	Lack of automated QA gates between the import module (REQ-002) and analysis pipeline (REQ-003).	Medium × Medium = High (9)	Automated test matrix (TEST-003) and image QC hooks inside the pipeline.	Low × Low = Low (1) Clinician-facing dashboard highlights residual warnings for manual acknowledgement.
RISK-003: Audit trail corruption	Investigators cannot reconstruct prior runs because audit entries were dropped.	Compliance breach or inability to support safety investigations.	Misconfigured storage or missing webhook callbacks when persisting audit artifacts from REQ-005.	Medium × Low = Low (3)	Continuous integration checks (TEST-003) and deployment checklist in docs/ius.md.	Low × Low = Low (1) Acceptable provided long-term storage health checks pass weekly.
RISK-004: Unauthorized workspace access	An unapproved clinician launches the UI and edits live studies without authentication.	Breach of patient confidentiality and risk of tampering with diagnostic evidence.	Misconfigured identity provider or bypass of the login screen covered in REQ-001.	High × Low = Medium (4)	OpenID Connect gateway, MFA, and TEST-001 regression runs.	Medium × Low = Low (3) Acceptable when the security team performs quarterly access recertification.
RISK-005: Notification backlog for identifier mismatches	Identifier reconciliation alerts pile up, so clinicians miss urgent mismatches.	Delayed detection of study/patient mix-ups (see REQ-005).	Downstream webhook queue saturation or operator console outages.	Medium × Medium = High (9)	Autoscaling webhook workers plus the workflow exercised by TEST-002.	Low × Medium = Low (3) Escalate to the on-call operator when more than five alerts remain untriaged for over 30 minutes.
RISK-006: Exported report mismatch	Generated PDF or DICOM SR exports omit the source dataset IDs.	Inability to justify clinical findings or to reconstruct the original evidence trail.	Faulty serialization of the output schema defined in REQ-004.	Medium × Low = Low (3)	Automated CI pipeline (TEST-003) that validates exports against golden files.	Low × Low = Low (1) Acceptable when QA signs off release bundles that include checksum manifests.
RISK-007: Pipeline drift between releases	Model weights or preprocessing steps change without validation, yielding inconsistent diagnoses.	False positives/negatives that could trigger incorrect therapy.	Manual edits to REQ-003 pipeline components without rerunning validation.	High × Medium = High (12)	Continuous integration gates in TEST-003 plus code reviews for every pipeline update.	Medium × Low = Low (3) Acceptable once deployment reports attach the validation pack and a signed change record.
RISK-008: Data retention policy breach	Backups omit audit logs, preventing reconstruction of historic identifier fixes.	Compliance violations with ICH-GCP and inability to support investigations.	Retention scripts ignore the storage bucket defined in REQ-005.	Medium × Medium = High (9)	Scheduled integrity checks plus TEST-003 to validate log export scripts.	Low × Low = Low (1) Acceptable when quarterly SOP reviews confirm retention evidence is archived.
RISK-009: Automated test evidence missing	Regression runs referenced by TEST-003 fail but their output is not included in the validation pack.	Release approvals rely on incomplete evidence trails.	Operator forgets to pass –playwright-dir or the CI job skips the autotest block.	Medium × Medium = High (9)	TraceFlow CLI guards that raise on missing artifacts plus the process captured in docs/ius.md.	Low × Low = Low (1) Acceptable provided release managers verify example.pdf before sign-off.
RISK-010: External interface misconfiguration	Downstream systems consume reconciliation webhooks with the wrong schema and silently drop events.	Identifier mismatches never reach clinical users even though TraceFlow generated the alerts.	Integration guides diverge from the specification in REQ-005 or design doc steps.	Medium × Medium = High (9)	Installation & User Specification walkthrough (docs/ius.md) and contract tests executed as part of TEST-002.	Low × Medium = Low (3) Acceptable once each release captures signed integration test outputs in the validation pack.
RISK-011: Manual test paperwork incomplete	Manual acceptance tests (TEST-001) are executed but signatures or pass/fail states are unsigned, invalidating the safety case.	Regulatory submissions may be rejected, delaying patient access.	Clinicians forget to fill the interactive PDF fields exported by TraceFlow.	Low × Medium = Low (3)	The test cover sheet plus electronic signature workflow baked into TraceFlow forms.	Low × Low = Low (1) Acceptable when QA audits confirm PDF form completion before release.

4 Requirements

These are some general notes on this set of requirements. Other content within subsections are for specific requirements.

4.1 Traceability Matrix

Req ID \ ***Test ID***	TEST-001	TEST-002	TEST-003
REQ-001	✓		
REQ-002	✓		
REQ-003		✓	✓
REQ-004			
REQ-005		✓	

4.2 REQ-001: User authentication

Test ID: TEST-001
Users must be able to authenticate using their email address and a password.

4.2.1 Example LaTeX equation

The strength of the user's password should follow the entropy equation:

$$H = L \times \log \_2(N)$$

Where H is the entropy, L is the password length, and N is the number of possible symbols.

4.2.2 Example inline code formatting

The ID of the user is, e.g., `1af345e6`.

4.3 REQ-002: MRI dataset import

Test ID: TEST-001
The platform must support importing MRI datasets in DICOM format.

4.3.1 Example image (.png)

Figure 2: VoxelFlow Logo

4.4 REQ-003: Image analysis pipeline

Test ID: TEST-002 **Test ID:** TEST-003

- The platform should provide a Python API for building image analysis pipelines.
- The pipelines must be able to process MRI datasets in a compliant way.

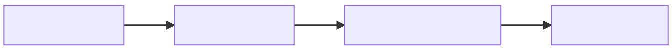

Figure 3:

4.4.1 Example flow chart (using mermaid)

4.4.2 Example code block

```python
def hello_world():
    print("Hello world!")
```

4.5 REQ-004: Output data

The platform must be able to export the results of image analysis pipelines in a standard format, including the following parameters:

DICOM Series	Key Parameters	Typical Parameters	Map to Image Type
T1 VFA	TR, TE, FA	96x96x24 FA=2°,17°,32°	`vfa`
High-res pre-contrast	TR, TE, FA	512x512x92 FA=32°	`high-res-pre`
T1-weighted dynamic	TR, TE, FA, Temporal resolution	96x96x24 FA=17° Temporal resolution: 2s to 10s	`dynamic-uncorrected`
High-res post-contrast	TR, TE, FA	512x512x92 FA=32°	`high-res-post`

4.6 REQ-005: Patient-study reconciliation

Test ID: TEST-002

The ingestion service shall ensure that study metadata stays associated with the correct patient identifier to reduce **RISK-001** and **RISK-002**.

- Cross-validate the incoming HL7 and DICOM identifiers before persisting them.
- Flag mismatches for operator review and capture evidence via TEST-002.
- Provide a reconciliation webhook so downstream systems can resynchronize identifiers.

4.6.1 Example sequence diagram

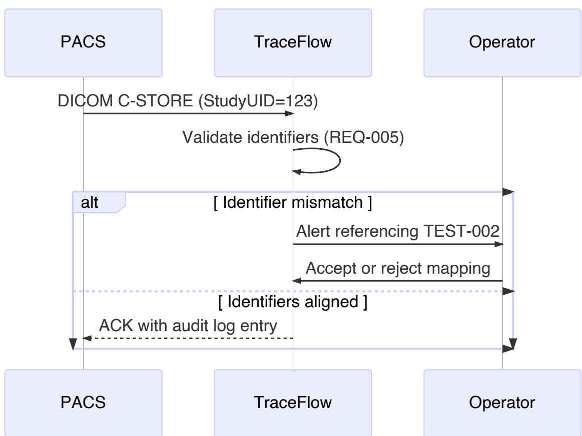

Figure 4:

5 Manual Acceptance Tests

These are some general notes on this set of test. Other content within subsections are for specific tests.

Tester	
Test Date	
Result	
Observations	

5.1 TEST-001: User authentication

Requirement ID: REQ-001 **Requirement ID:** REQ-002

5.1.1 Test Steps:

- Navigate to the login page.
- Enter a valid email address and password.
- Click the "Login" button.

5.1.2 Expected Result:

The user is logged in and redirected to the main dashboard.

5.1.3 Test Outcome:

Pass ☐ **Fail** ☐ **Skip** ☐

Comments

5.2 TEST-002: Automatic Playwright test

Requirement ID: REQ-003 **Requirement ID:** REQ-005

5.2.1 Test Steps:

- The `autoplaywright` block runs the Playwright sample located in the `playwright/` directory and captures the video, stdout/stderr, and a 3×3 key frame grid from the test execution.
- The scripted run intentionally drives a patient-identifier edit workflow to demonstrate the mitigation for **RISK-001** .

5.2.2 Expected Result:

The test should pass successfully.

5.2.3 Test Outcome:

Pass ☑ **Fail** ☐ **Skip** ☐

Playwright Test: 0001-test-counter

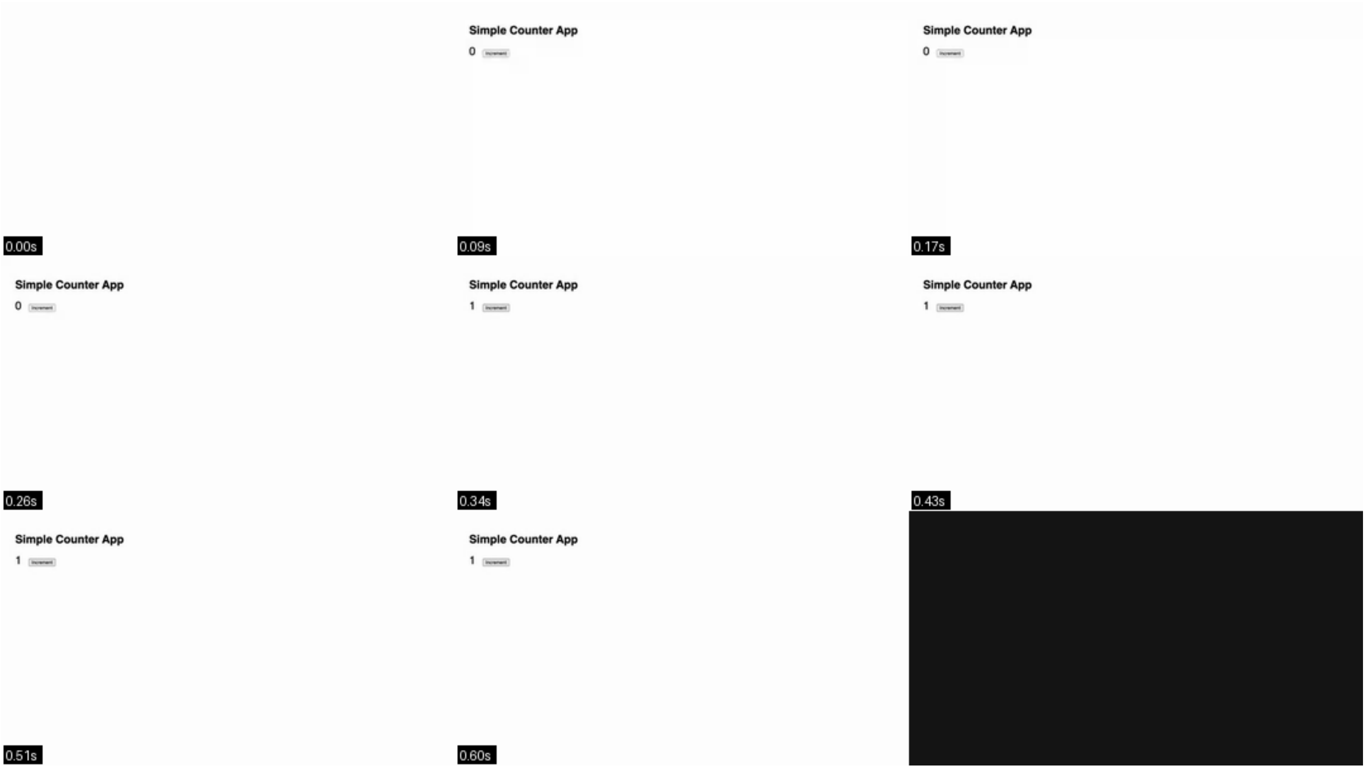

Figure 5: Key frames captured from 0001-test-counter

```
Running 1 test using 1 worker

  [PASS]  1 counter.spec.js:5:1 > 0001-test-counter (1.5s)

  1 passed (2.4s)
```

5.3 TEST-003: Automatic test

Requirement ID: REQ-003

5.3.1 Test Steps:

- This `autotest` block demonstrates running a simple pytest command as part of the report.
- The quick-running suite doubles as a guardrail for **RISK-002** and **RISK-003** ; failures block deployments.

5.3.2 Expected Result:

The test should pass successfully.

5.3.3 Test Outcome:

Pass ☑ **Fail** ☐ **Skip** ☐
Playwright Test: 0001-test-counter

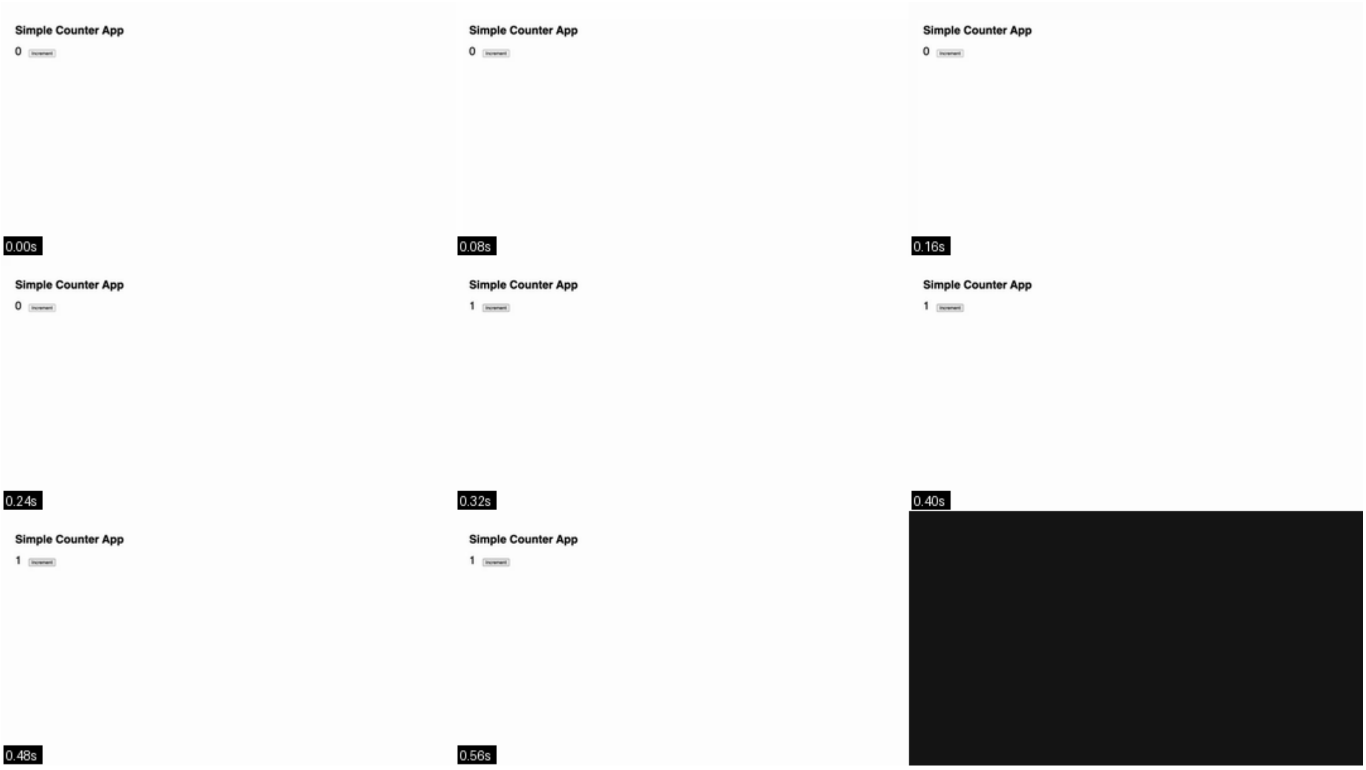

Figure 6: Key frames captured from 0001-test-counter

```
Running 1 test using 1 worker

  [PASS]  1 counter.spec.js:5:1 > 0001-test-counter (801ms)

  1 passed (1.3s)
```

Pass ☑ **Fail** ☐ **Skip** ☐

```
============================= test session starts ==============================
platform darwin -- Python 3.11.13, pytest-9.0.1, pluggy-1.6.0 --
    /Users/matthew.heaton/Work/SoftwareDevelopment/TraceFlow/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /Users/matthew.heaton/Work/SoftwareDevelopment/TraceFlow
configfile: pyproject.toml
collecting ... collected 1 item

tests/test_hello_world.py::TestHelloWorld::test_zero Hello world!
PASSED

============================== 1 passed in 0.00s ===============================
```